

5. Herramientas y procesos de construcción

Editores e IDEs · Git · Construcción y dependencias · Calidad · Depuración · Entornos · Docker · Automatización y CI

5.1 Editores e IDEs

Un editor de código es donde escribes el código. Un IDE (Integrated Development Environment) es un editor con todo lo demás integrado: ejecutar, depurar, refactorizar, gestionar dependencias. La diferencia práctica: con un editor cambias de herramienta para compilar o depurar; con un IDE todo está en el mismo sitio.

Aspecto	Editor de código	IDE
Peso y velocidad	Ligero, arranca rápido	Más pesado, carga el proyecto al abrir
Funciones	Edición, autocompletado, extensiones	Build, debug, pruebas, refactorizaciones integradas
Conviene cuando	Varios lenguajes, proyectos pequeños o scripts	Proyectos medianos/grandes con un lenguaje principal
Ejemplos	VS Code, Sublime Text, Vim	IntelliJ IDEA (Java/Kotlin), Eclipse (Java), PyCharm (Python)

Situación típica: empiezas con Java. Podrías usar VS Code con extensiones, pero IntelliJ te avisa de errores mientras escribes, genera getters/setters y permite depurar sin salir del editor. Para Java de cierto tamaño, el IDE ahorra mucho tiempo.

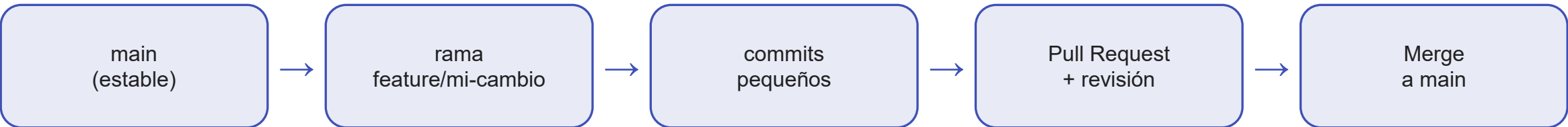
Buenas prácticas: abre siempre la CARPETA RAÍZ del proyecto (no un archivo suelto) · activa el formato al guardar · define tareas del proyecto (build, test, lint) para que todos las usen igual.

5.2 Control de versiones (Git) — conceptos

¿Qué problema resuelve? Sin Git, dos personas que tocan el mismo archivo se sobrescriben, y si rompes algo no hay vuelta atrás. Git guarda el HISTORIAL COMPLETO y permite RAMAS PARALELAS. Como el historial de guardados de un videojuego: vuelves a un punto anterior o exploras un camino alternativo sin perder el original.

Concepto	Qué es
Repositorio	La carpeta del proyecto con todo su historial. Local (tu ordenador) o remoto (GitHub, GitLab...)
Commit	Una "foto" del proyecto en un momento concreto, con un mensaje que explica qué cambió y por qué
Branch (rama)	Una línea de trabajo paralela: desarrollas una feature sin tocar main
Merge	Unir una rama de vuelta a main una vez revisada y aprobada
Pull Request (PR)	Solicitud de revisión antes del merge: el equipo revisa los cambios y da el visto bueno

Flujo habitual (GitHub Flow)



5.2 Git — .gitignore, errores habituales y mensajes

.gitignore: si Git puede regenerarlo, ignóralo

```
target/           # compilación de Java (se regenera con mvn
package)
node_modules/     # dependencias de Node (npm install)
.env              # contraseñas locales — ¡nunca al repo!
.idea/ *.iml      # configuración personal del IDE
```

Sin .gitignore es fácil subir por accidente contraseñas, carpetas enormes generadas automáticamente o configuración personal del editor.

Error habitual	Consecuencia
Trabajar directamente en main sin ramas	Si algo falla, contaminas el código estable
Commits gigantes con mensaje "cambios"	Imposible entender qué pasó después
No hacer pull antes de empezar	Conflictos evitables
Subir el .env con contraseñas	Problema GRAVE de seguridad

Mensajes de commit: describe QUÉ cambió y POR QUÉ, no el cómo (eso lo dice el código). Bien: "Añadir validación de email en el formulario de registro" — Mal: "cambios", "fix", "wip".

5.3 Sistemas de construcción y gestores de dependencias

Cuando el proyecto crece, compilar a mano (javac Fichero.java) no escala. El sistema de construcción automatiza todo con un comando: compila, pasa los tests, empaqueta. El gestor de dependencias descarga las librerías externas en la versión exacta que necesitas — sin él, descargarías cada JAR a mano.

```
<!-- pom.xml: declaras qué necesitas y
Maven lo descarga -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
</dependency>

mvn test      → compila y pasa los tests
mvn package   → genera el JAR final
```

Lenguaje	Construcción	Dependencias	Config
Java	Maven / Gradle	Maven Central	pom.xml / build.gradle
JS / Node	npm scripts / Vite	npm / pnpm / yarn	package.json
Python	tox	pip / poetry	requirements.txt / pyproject.toml
C / C++	Make / CMake	vcpkg / Conan	Makefile / CMakeLists.txt

Centraliza las tareas: define comandos estándar (build, test, lint) en la configuración del proyecto, para que todo el equipo ejecute exactamente lo mismo. Evita el clásico "en mi PC funciona".

5.4 Calidad de código

Un código puede funcionar y aun así ser difícil de leer, mantener o cambiar. Si nadie controla la calidad, el proyecto acaba siendo algo que nadie quiere tocar. Estas herramientas detectan los problemas automáticamente, antes de producción:

Herramienta	Qué hace	Ejemplo de detección	Herramientas por lenguaje
Linters	Analiza el código en busca de errores comunes y malas prácticas SIN ejecutarlo	Variables declaradas y nunca usadas; comparaciones siempre falsas	Checkstyle, PMD (Java) · ESLint (JS) · flake8, pylint (Python)
Formateador	Aplica un estilo uniforme automáticamente (indentación, espacios, llaves)	Todo el equipo produce código con el mismo aspecto	Spotless (Java) · Prettier (JS) · Black (Python)
Análisis estático	Va más allá del linter: vulnerabilidades, dependencias obsoletas, bugs sutiles	Concatenar datos del usuario en una consulta SQL (inyección SQL)	SonarQube, Semgrep, Snyk

Orden recomendado: formato → linter → pruebas. El formato y el linter tardan segundos; las pruebas pueden tardar más. Si el linter falla, no esperes a que acaben todas las pruebas.

5.5 Depuración

El primer instinto es llenar el código de `System.out.println`. Funciona para casos simples, pero el depurador (debugger) permite PAUSAR la ejecución en cualquier punto, inspeccionar cada variable en ese momento exacto y avanzar línea a línea hasta ver dónde se tuerce la lógica.

Control	Atajo (IntelliJ)	Qué hace
Step over	F8	Ejecuta la línea actual y para en la siguiente; las llamadas a función se ejecutan completas sin entrar
Step into	F7	Si la línea llama a una función, entra dentro para verla paso a paso
Resume	F9	Continúa hasta el siguiente breakpoint (o hasta el final)

Paso a paso en IntelliJ

1. Coloca un breakpoint con clic en el margen izquierdo (punto rojo).
2. Arranca en modo debug (icono del insecto o Shift+F9): el programa se detiene en el breakpoint.
3. En el panel inferior verás las variables locales con sus valores: comprueba si son los que esperas.
4. Step over línea a línea; si una función parece sospechosa, Step into.
5. Encuentra el error, corrígelo, quita el breakpoint y vuelve a ejecutar.

Buenas prácticas: breakpoint JUSTO ANTES del punto sospechoso · escribe antes en papel el valor que esperas en cada variable (si no coincide, ahí está el error) · un `println` en producción es un olvido esperando ocurrir. Y si el programa funciona pero va lento: PERFILADO (profiling) — medir cuánto tarda cada parte (avanzado, no en primero).

5.6 Entornos y configuración

Entorno	Dónde	Para qué
Desarrollo (dev)	Tu ordenador	Programar y probar localmente
Pruebas (staging)	Un servidor de pruebas	Validar antes de publicar; puede contener datos ficticios
Producción (prod)	El servidor real	Lo que usan los usuarios reales; no se toca sin control

El código es el MISMO en los tres entornos; lo que cambia son los valores de configuración: URL de la base de datos, contraseñas, servidor de correo...

Variables de entorno y el archivo .env

```
# .env – solo en tu ordenador, nunca al repositorio
DB_HOST=localhost
DB_PORT=5432
DB_PASSWORD=contraseña_local_solo_desarrollo

// El código lee la variable, no el valor:
String host = System.getenv("DB_HOST");
```

Una variable de entorno es un par clave-valor que la app lee al arrancar, sin que el valor esté en el código. Cambias el comportamiento sin tocar el código ni recompilar.

En producción, las mismas variables tienen otros valores configurados en el servidor.

Reglas: NUNCA contraseñas hardcodeadas en el código · .env siempre en .gitignore · si falta una variable al arrancar, el programa debe fallar con mensaje claro, no continuar con un valor vacío.

5.7 Contenedores y virtualización (Docker)

¿Qué problema resuelven? El clásico "en mi ordenador funciona". Un contenedor lleva la app JUNTO CON TODO lo que necesita (sistema base, librerías, runtime) para ejecutarse igual en cualquier máquina.

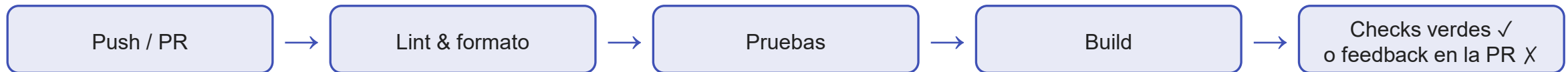
Concepto	Qué es
Imagen	Plantilla de solo lectura con la app y sus dependencias. Una receta versionada (miapp:1.0)
Contenedor	Instancia en ejecución de una imagen. Como un táper creado a partir de la receta: lo usas y lo borras sin afectar a la imagen
Registry	Almacén donde publicas y descargas imágenes (Docker Hub, GHCR...)

Característica	Contenedor	Máquina virtual
Arranque	Segundos	Decenas de segundos / minutos
Tamaño	Ligero (MB – cientos de MB)	Pesado (GB)
Aislamiento	A nivel de proceso (kernel compartido)	Kernel propio (aislamiento más fuerte)
Uso típico	Empaquetar apps y servicios	Emular sistemas completos

5.8 Automatización de tareas y CI

Objetivo: que los pasos importantes (formato, análisis, pruebas, build) se hagan SIEMPRE IGUAL y AUTOMÁTICAMENTE. Tres piezas que se complementan:

Pieza	Qué es
Scripts	Comandos con nombre dentro del proyecto: build, test, lint, format
Hooks de Git	Reglas que se ejecutan antes/después de ciertas acciones. Un hook pre-commit puede pasar el linter y las pruebas rápidas ANTES de confirmar el commit
CI (Integración Continua)	Servicio en la nube que, en cada push o PR, clona el repositorio y ejecuta automáticamente todos los checks



```
# .github/workflows/ci.yml - GitHub Actions
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # descarga el código
      - uses: actions/setup-java@v4        # configura Java 17
      - run: mvn test                       # el mismo comando que en local
```

Si mvn test falla, la PR queda en rojo y no se puede fusionar.

Consejos: scripts cortos y autoexplicativos · primero lo rápido (lint), luego lo costoso (pruebas) · secretos en el almacén del proveedor, no en el repo · cachea dependencias.

Tabla resumen del apartado

Tema	¿Qué es?	¿Para qué sirve?
Editores / IDEs	Herramientas para escribir y gestionar código	Producir y comprender código con ayudas integradas
Control de versiones	Historial, ramas y fusiones	Trabajar en equipo sin perder cambios
Construcción / Dependencias	Proceso y bibliotecas del proyecto	Obtener ejecutables y traer librerías externas
Calidad	Linter, formateador, análisis, cobertura	Mantener estilo, detectar fallos, medir pruebas
Depuración	Pausar y explorar la ejecución	Encontrar errores sin tocar el código
Entornos / Config	Valores por entorno y secretos	Cambiar comportamiento sin tocar el código
Contenedores	Imágenes y contenedores aislados	Ejecutar igual en cualquier máquina
Automatización / CI	Scripts, hooks y verificación en servidor	Estandarizar pasos y detectar problemas pronto